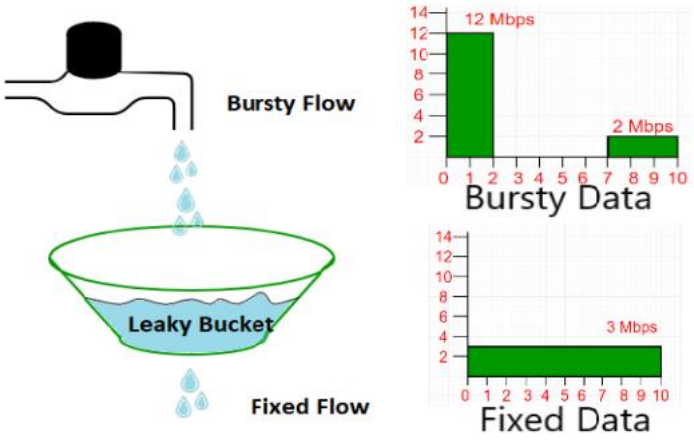
Experiment 8

Aim: Write a program for congestion control using Leaky bucket algorithm

In the network layer, before the network can make Quality of service guarantees, it must know what traffic is being guaranteed. One of the main causes of congestion is that traffic is often bursty. To understand this concept first we have to know little about traffic shaping. Traffic Shaping is a mechanism to control the amount and the rate of traffic sent to the network. Approach of congestion management is called Traffic shaping. Traffic shaping helps to regulate the rate of data transmission and reduces congestion.

There are 2 types of traffic shaping algorithms: 1. Leaky Bucket 2. Token Bucket

Suppose we have a bucket in which we are pouring water, at random points in time, but we have to get water at a fixed rate, to achieve this we will make a hole at the bottom of the bucket. This will ensure that the water coming out is at some fixed rate, and also if the bucket gets full, then we will stop pouring water into it. The input rate can vary, but the output rate remains constant. Similarly, in networking, a technique called leaky bucket can smooth out bursty traffic. Bursty chunks are stored in the bucket and sent out at an average rate.



In the above figure, we assume that the network has committed a bandwidth of 3 Mbps for a host. The use of the leaky bucket shapes the input traffic to make it conform to this commitment. In the above figure, the host sends a burst of data at a rate of 12 Mbps for 2s, for a total of 24 Mbits of data. The host is silent for 5 s and then sends data at a rate of 2 Mbps for 3 s, for a total of 6 Mbits of data. In all, the host has sent 30 Mbits of data in 10 s. The leaky bucket smooths out the traffic by sending out data at a rate of 3 Mbps during the same 10 s. Without the leaky bucket, the beginning burst may have hurt the network by consuming more bandwidth than is set aside for this host. We can also see that the leaky bucket may prevent congestion.

A simple leaky bucket algorithm can be implemented using FIFO queue. A FIFO queue holds the packets. If the traffic consists of fixed-size packets (e.g., cells in ATM networks), the process removes a fixed number of packets from the queue at each tick of the clock. If the traffic consists of variable-length packets, the fixed output rate must be based on the number of bytes or bits.

The following is an algorithm for variable-length packets:

1. Initialize a counter to n at the tick of the clock.

2. Repeat until n is smaller than the packet size of the packet at the head of the queue.

    1. Pop a packet out of the head of the queue, say P.

    2. Send the packet P, into the network

    3. Decrement the counter by the size of packet P.

3. Reset the counter and go to step 1.

Ex: Initial Setup

- **Clock Rate Credit (n):** 1000 bytes per clock tick
- **Initial Queue:**

[Tail] [200 | 700 | 500 | 450 | 400 | 200] [Head]

Tick 1 ($n = 1000$)

1. **Packet 1 (Size: 200 at the head):**
    - Check: $n \geq 200$ ($1000 \geq 200$).
    - Action: Pop and send packet of size **200** into the network.
    - Remaining credit: $n = 1000 - 200 = 800$.
    - Queue remaining:

[200 | 700 | 500 | 450 | 400]

2. **Packet 2 (Size: 400 at the head):**
    - Check: $n \geq 400$ ($800 \geq 400$).
    - Action: Pop and send packet of size **400** into the network.
    - Remaining credit: $n = 800 - 400 = 400$.
    - Queue remaining:

[200 | 700 | 500 | 450]

3. **Packet 3 (Size: 450 at the head):**
    - Check: $n \geq 450$? No, because $400 < 450$.
    - The remaining counter capacity is insufficient to send the entire packet, so transmission stops for this tick.

How Remaining Packets are Sent in Subsequent Ticks

"Initialise $n = 1000$ *on another tick of the clock. This procedure is repeated until all the packets are sent into the network."*

Tick 2 (n resets to 1000)

- Current Queue: [200 | 700 | 500 | 450]

1. **Packet 3 (Size: 450 at the head):**
    - Check: $1000 \geq 450$.
    - Action: Send packet **450**.
    - Remaining credit: $n = 1000 - 450 = 550$.
    - Queue remaining: [200 | 700 | 500].
2. **Packet 4 (Size: 500 at the head):**
    - Check: $550 \geq 500$.
    - Action: Send packet **500**.
    - Remaining credit: $n = 550 - 500 = 50$.
    - Queue remaining: [200 | 700].
3. **Packet 5 (Size: 700 at the head):**
    - Check: Is $50 \geq 700$? No ($50 < 700$).
    - Transmission stops for Tick 2.

Tick 3 (n resets to 1000)

- Current Queue: [200 | 700]

1. **Packet 5 (Size: 700 at the head):**
    - Check: $1000 \geq 700$.
    - Action: Send packet **700**.
    - Remaining credit: $n = 1000 - 700 = 300$.
    - Queue remaining: [200].
2. **Packet 6 (Size: 200 at the head):**
    - Check: $300 \geq 200$.
    - Action: Send packet **200**.
    - Remaining credit: $n = 300 - 200 = 100$.

- Queue remaining: [ ] (Empty).

Clock Tick	Credit (n)	Packets Sent from Head	Bytes Sent	Leftover Credit	Remaining Queue
Tick 1	1000 PDF	200, then 400 PDF	600	400 PDF	[200 | 700 | 500 | 450] PDF
Tick 2	1000	450, then 500	950	50	[200 | 700]
Tick 3	1000	700, then 200	900	100	[ ] (All packets transmitted)

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#define MIN(x, y) (((x) > (y)) ? (y) : (x))
#define MAX_TIME 100
int main() {
    int cap, orate;
    int inp[MAX_TIME] = {0};
    int i = 0, nsec = 0, ch = 0;
    int count = 0;   // Current buffer level
    int drop = 0;    // Dropped packets in current tick
    int sent = 0;    // Packets sent in current tick
    int incoming = 0;
    printf("\nEnter Bucket Size: ");
    if (scanf("%d", &cap) != 1) return 1;
    printf("Enter Output Rate: ");
    if (scanf("%d", &orate) != 1) return 1;
    do {
        if (i >= MAX_TIME) {
            printf("\nMaximum time limit reached.\n");
```

```
            break;
        }
        printf("\nEnter number of packets coming at second %d: ", i + 1);
        scanf("%d", &inp[i]);
        i++;
        printf("Enter 1 to continue or 0 to quit: ");
        scanf("%d", &ch);
    } while (ch);
    nsec = i;
    printf("\nSecond\tReceived\tSent\tDropped\tRemained\n");
    printf("-------------------------------------------------\n");
    for (i = 0; count > 0 || i < nsec; i++) {
        // After nsec seconds, incoming packets are 0
        incoming = (i < nsec) ? inp[i] : 0;
        // Step 1: Add incoming packets and check for overflow
        if (incoming + count > cap) {
            drop = (incoming + count) - cap;
            count = cap; // Bucket is filled to capacity
        } else {
            drop = 0;
            count += incoming;
        }
        // Step 2: Leak/transmit packets at the specified output rate
        sent = MIN(count, orate);
        count -= sent;
        printf("%d\t%d\t\t%d\t%d\t%d\n", i + 1, incoming, sent, drop, count);
    }
    return 0;
}
```

Output:

```
Enter Bucket Size: 50
Enter Output Rate: 10

Enter number of packets coming at second 1: 10
Enter 1 to continue or 0 to quit: 1

Enter number of packets coming at second 2: 20
Enter 1 to continue or 0 to quit: 1

Enter number of packets coming at second 3: 30
Enter 1 to continue or 0 to quit: 1

Enter number of packets coming at second 4: 40
Enter 1 to continue or 0 to quit: 0

Second  Received        Sent    Dropped Remained
------------------------------------------------
1       10              10      0       0
2       20              10      0       10
3       30              10      0       30
4       40              10      20      40
5       0               10      0       30
6       0               10      0       20
7       0               10      0       10
8       0               10      0       0
```